AI Assignment 1

Adult Income Dataset – Data Preparation and Exploratory Data Analysis

This report documents a complete and reproducible workflow for preparing and exploring the Adult Income dataset. It focuses on professional data handling practices, including missing-value treatment, basic outlier inspection, feature engineering (encoding and scaling), and visual exploration of key relationships. The document is structured as a self-contained technical report for a binary classification assignment in Artificial Intelligence.

Each preparation step explicitly lists the main Python / scikit-learn function used and explains its role on the same line. Whenever a step produces a figure, the corresponding visualization is shown directly below that step together with a concise caption. This style matches the assignment requirement of clearly connecting code, rationale, and results.

Project Type: Binary Classification

Dataset: Adult Income (Kaggle / UCI)

1. Dataset Overview

The Adult Income dataset contains demographic and employment information for 48842 individuals. Each record describes a person using attributes such as age, education level, marital status, occupation, hours worked per week, and a binary income label indicating whether the person earns more than 50,000 USD per year. The objective of this assignment is to prepare a clean, model-ready feature matrix and to analyze the dataset through descriptive statistics and visual exploration, forming the foundation for supervised learning models.

Dataset summary

- Number of instances (rows): 48842
- Number of input features (excluding the target): 14
- Target column: income
- Class distribution:
  - <=50K: 37155 instances (76.07%)
  - >50K: 11687 instances (23.93%)
- Balance ratio (minority / majority): 0.31
- Typical age: mean 38.6 years (median 37), range 17–90 years.
- Typical working time: mean 40.4 hours/week (median 40), range 1–99 hours/week.

2. Step-by-Step Data Preparation

• **Step 1 – Load the dataset:** pd.read_csv("adult.csv") – reads the raw CSV file from disk into a pandas DataFrame called data, providing a structured table for all downstream processing.

• **Step 2 – Inspect structure and basic statistics:** data.info(), data.describe(include="all") – reports the number of observations, data types, and core descriptive statistics, giving an initial assessment of data quality and feature characteristics.

• **Step 3 – Standardize missing values and string formatting:** data.replace("?", np.nan) and data[col] = data[col].str.strip() – converts the placeholder "?" into proper missing values (NaN) and removes leading/trailing spaces from text fields, ensuring consistency for later encoding and imputation.

• **Step 4 – Quantify missingness per feature:** data.isna().sum().sort_values(ascending=False) – computes the number of missing entries in each column and orders features by their missingness, guiding the design of the imputation strategy.

• **Step 5 – Split features and target label:** X = data.drop("income", axis=1), y = data["income"] – constructs the design matrix X (input features) and the target vector y, which is the standard interface for supervised learning methods.

• **Step 6 – Stratified train/test split:** train_test_split(X, y, test_size=0.2, stratify=y, random_state=42) – divides the dataset into training (80%) and test (20%) partitions, preserving the original class proportions so that evaluation metrics remain representative.

• **Step 7 – Identify numeric and categorical feature groups:** num_cols = X_train.select_dtypes(include=[np.number]).columns.tolist(), cat_cols = X_train.select_dtypes(include=["object"]).columns.tolist() – determines which columns require numeric preprocessing and which require categorical encoding.

• **Step 8 – Numeric preprocessing pipeline (imputation + scaling):** numeric_tf = Pipeline(steps=[("imputer", SimpleImputer(strategy="median")), ("scaler", StandardScaler())]) – fills missing numeric values with the median and standardizes all numeric features to zero mean and unit variance.

• **Step 9 – Categorical preprocessing pipeline (imputation + one■hot encoding):** categorical_tf = Pipeline(steps=[("imputer", SimpleImputer(strategy="most_frequent")), ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))]) – replaces missing categorical entries with the most frequent category and converts each category into a set of binary indicator (one■hot) features.

• **Step 10 – Column■wise combination of pipelines:** preprocess = ColumnTransformer(transformers=[("num", numeric_tf, num_cols), ("cat", categorical_tf, cat_cols)]) – merges the numeric and categorical pipelines into a single object that automatically applies the appropriate transformations to each subset of columns.

• **Step 11 – Fit and transform train and test sets:** Xt_train = preprocess.fit_transform(X_train), Xt_test = preprocess.transform(X_test) – learns imputation, scaling, and encoding parameters from the training data and applies them consistently to both training and test sets, producing a clean numeric feature matrix without missing values.

• **Step 12 – Feature engineering summary:** the combination of SimpleImputer, StandardScaler, and OneHotEncoder within Pipeline and ColumnTransformer constitutes the feature■engineering stage, transforming heterogeneous raw attributes into a homogeneous, model■ready representation.

3. Exploratory Data Analysis (EDA)

• **Step 13 – Visualize target class distribution:** data["income"].value_counts(normalize=True).mul(100).plot(kind="bar") – computes the percentage of each income class and produces a bar chart to diagnose imbalance in the target labels.

Figure 1 – Income class distribution, showing that the <=50K class is substantially more frequent than the >50K class.

• **Step 14 – Inspect age distribution:** data["age"].plot(kind="hist", bins=40) – plots a histogram of the age feature to understand the demographic profile of the dataset.

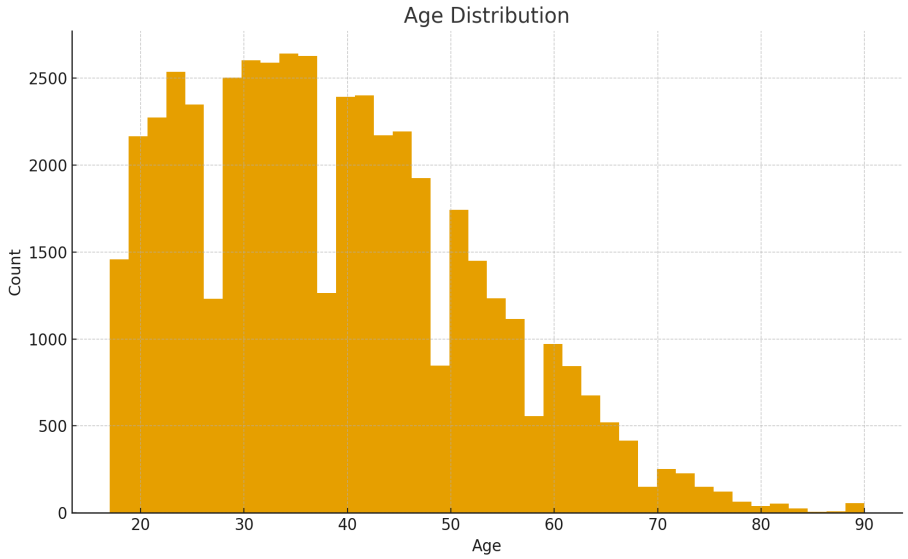

Figure 2 – Distribution of age, concentrated around typical working ages, with relatively few very young or very old individuals.

• **Step 15 – Relate marital status to income:** ct = pd.crosstab(data["marital-status"], data["income"], normalize="index"); ct.plot(kind="bar", stacked=True) – builds a normalized cross■tabulation between marital status and income and plots stacked bars to show, for each marital status category, the proportion of low■ and high■income individuals.

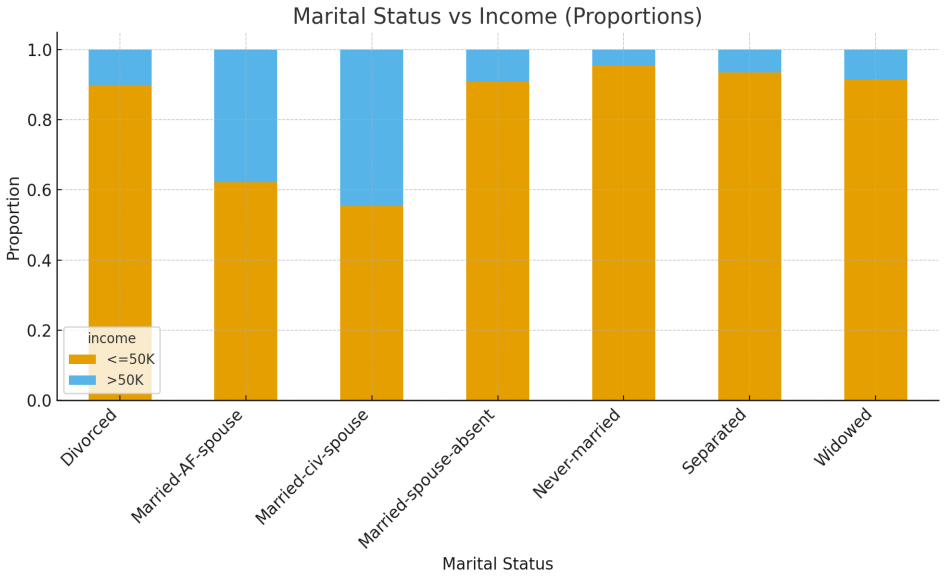

Figure 3 – Marital status versus income; Married-civ-spouse exhibits a noticeably higher proportion of individuals earning >50K.

• **Step 16 – Average working hours by education level:** data.groupby("education")["hours-per-week"].mean().sort_values(ascending=False).plot(kind="bar") – computes the mean weekly working hours for each education category and visualizes the results in a bar chart.

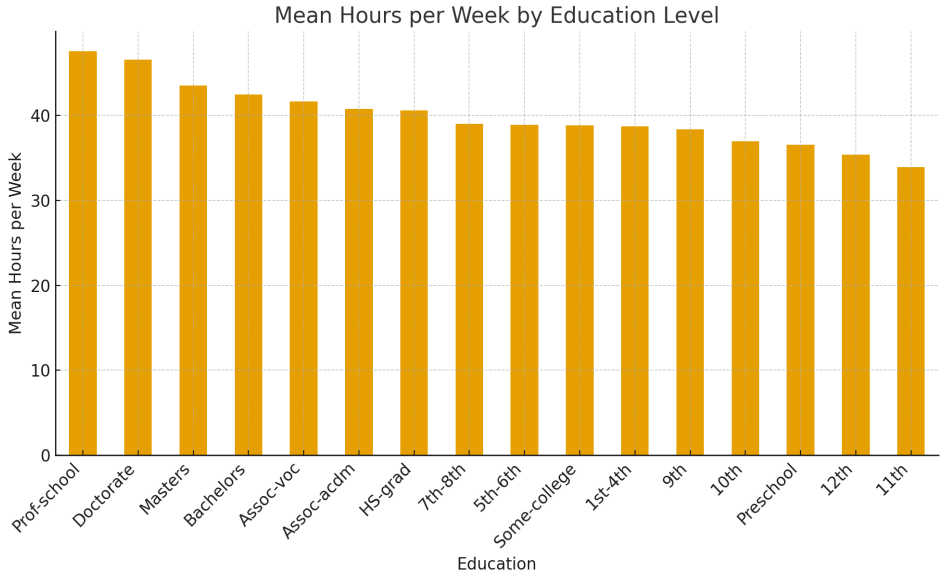

Figure 4 – Mean hours■per■week for each education level, indicating systematic differences in working time across education groups.

• **Step 17 – Examine hours■per■week distribution:** data["hours-per-week"].plot(kind="hist", bins=40) – draws a histogram of weekly working hours to identify typical workloads and potential extreme values.

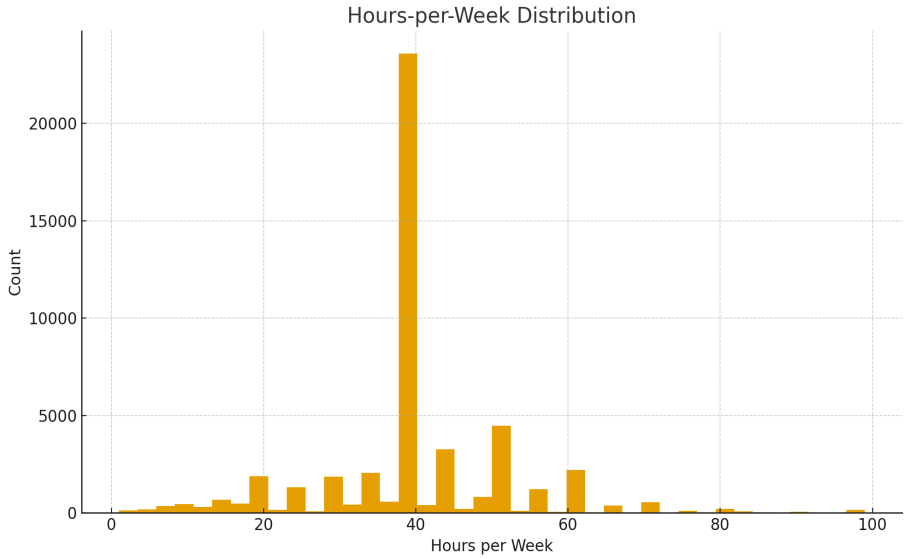

Figure 5 – Distribution of weekly working hours, with a dominant peak around a standard full■time schedule.

• **Step 18 – Education frequency (top 10 categories):** top_edu = data["education"].value_counts().head(10); top_edu.plot(kind="bar") – identifies the ten most frequent education categories and displays their counts to summarize the educational composition of the sample.

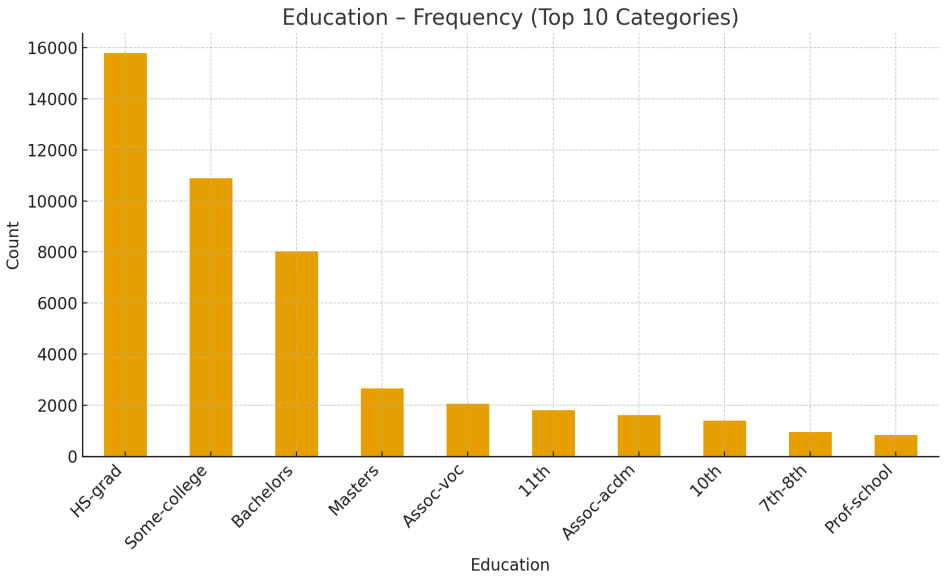

Figure 6 – Frequency of the top ten education categories in the dataset.

• **Step 19 – Correlation analysis for numeric features:** corr = data[num_cols].corr() followed by a heatmap using plt.imshow(corr) – computes the Pearson correlation matrix between all numeric variables and visualizes it, helping to identify strong linear relationships (for example, between education level and capital■gain).

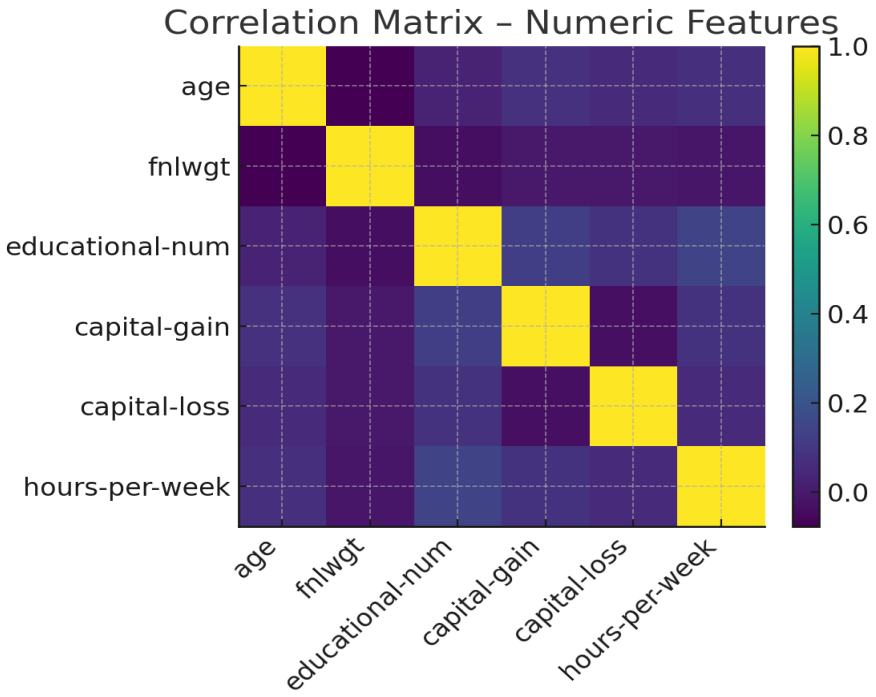

Figure 7 – Correlation matrix for numeric features, where darker cells indicate stronger positive or negative linear associations.

• **Step 20 – Outlier inspection (hours■per■week):** plt.boxplot(data["hours-per-week"]) – creates a boxplot for weekly working hours to visually detect potential outliers, such as extremely low or extremely high working times compared with the interquartile range.

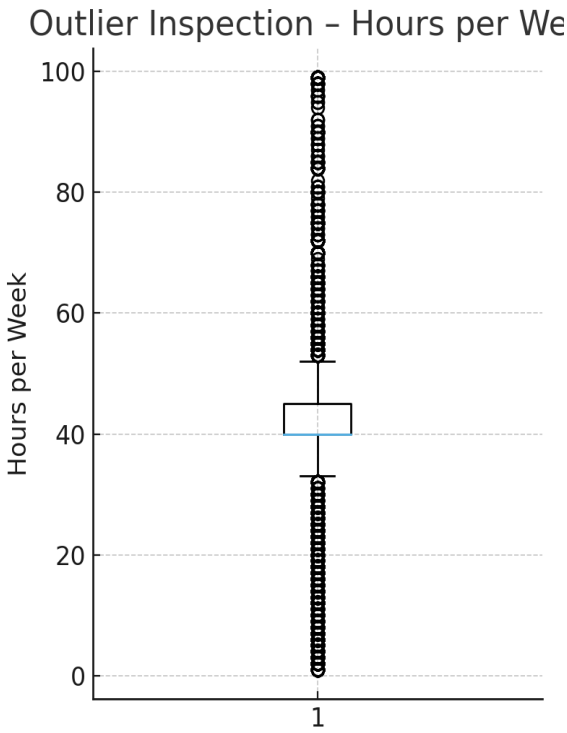

Figure 8 – Boxplot of hours■per■week; a small number of extreme values are visible, but they are retained as they may correspond to genuine work patterns.

4. Summary of Methods and Functions

The workflow relies on a focused set of functions and classes from pandas, NumPy, matplotlib, and scikit■learn: • **pd.read_csv()** – loading the dataset from a CSV file into a DataFrame.
• **DataFrame.info()**, **DataFrame.describe()**, **DataFrame.isna()** – assessing structure, basic statistics, and missing values.
• **train_test_split()** – generating stratified training and test splits.
• **SimpleImputer** – robust handling of missing numeric and categorical values.
• **StandardScaler** – scaling numeric attributes to comparable ranges.
• **OneHotEncoder** – converting categorical variables into a numerical one■hot representation.
• **Pipeline** and **ColumnTransformer** – encapsulating preprocessing logic into reusable and composable objects.
• **corr()** (pandas) and **plt.imshow()** – computing and visualizing the correlation structure among numeric features.
• **Histograms**, **bar charts**, and **boxplots** in matplotlib – supporting visual EDA and outlier inspection.

5. Key Insights and Next Steps

The completed data preparation and EDA pipeline yields several important conclusions: • The target variable is moderately imbalanced: 76.07% of individuals earn <=50K, while 23.93% earn >50K (Figure 1), so evaluation metrics should account for class imbalance.

• Age and hours■per■week follow plausible working■population distributions (Figures 2 and 5), providing confidence in the realism of the data.

• Marital status and education are clearly associated with income levels (Figures 3, 4, and 6), suggesting that they will be strong predictors in any classification model.

• The correlation matrix (Figure 7) reveals structured relationships among numeric features, which is useful when interpreting models or designing additional feature engineering steps.

• Outlier inspection (Figure 8) shows a limited number of extreme working hours; these are retained to preserve genuine behavioral variability, but they are clearly documented. The next phase of the assignment would be to attach one or more classification algorithms (e.g., logistic regression, decision tree, random forest, or gradient boosting) to the preprocess transformer and evaluate their performance using accuracy, precision, recall, F1■score, and calibration metrics, while explicitly addressing class imbalance through class weighting or resampling if necessary.